

Universitatea Ovidius din Constanta
Facultatea de Matematica si Informatica
Specializarea Informatica

Ingineria sistemelor software

Curs 7

Conf. univ. dr. Crenguta M. Puchianu
2020



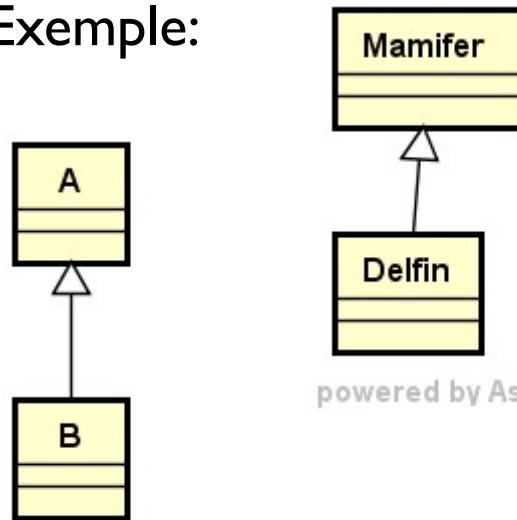
Analiza cerintelor software. Modelul domeniului



Relatia de generalizare

- Relație tranzitivă, antisimetrică între două clase, în care una este supraclassa celeilalte numita subclasă, în sensul că subclasa moștenește elementele de modelare ale suprac clasei sale.
- O supraclassă poate avea mai multe subclase, caz în care supraclassa este descrierea unei mulțimi de instanțe (indirecte) cu caracteristici comune tuturor subclaselor sale.

- Exemple:



powered by Astah

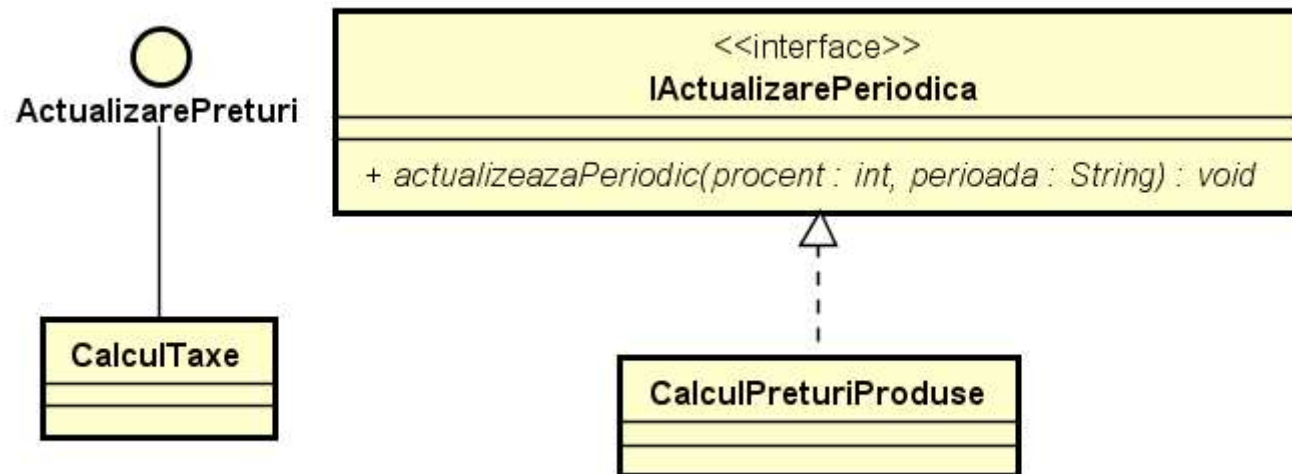
powered by Astah

ActualizarePreturi



Relatia de realizare

- Relație între o specificație (interfață sau caz de utilizare) și o implementare a sa (clasă sau colaborare).
- Exemple:

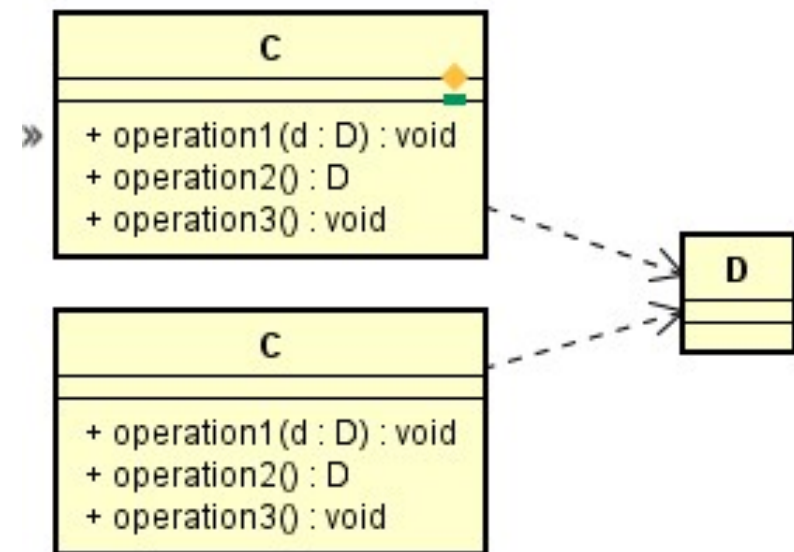
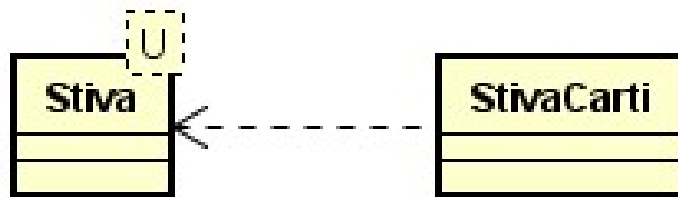


powered by Astah



Relatia UML de dependenta

- Relație între două clase (elemente de modelare) în care o modificare a unei clase (furnizor) poate afecta sau furniza informația necesară altei clase.
- Tipuri de dependență:
 - abstractizare
 - legare
 - permisiune și
 - folosire.
- Exemple:



Tehnica de construire a diagramei UML de clase a domeniului

- Pas 1. Gasirea substantivelor
- Pas 2. Evaluarea substantivelor si transformarea lor in clase sau obiecte, attribute sau valori de atribut
- Pas 3. Analiza claselor – numai clasele care apartin domeniului problemei
- Pas 4. Analiza rolurilor

Rol = este o descriere a unei multimi de proprietati si actiuni pe care le au sau le executa instantele unui alt concept care joaca rolul respectiv intr-un anumit context.

Rol => o clasa ale carei instante joaca rolul respectiv

Exp: /Student:Persoana, /Profesor:Persoana

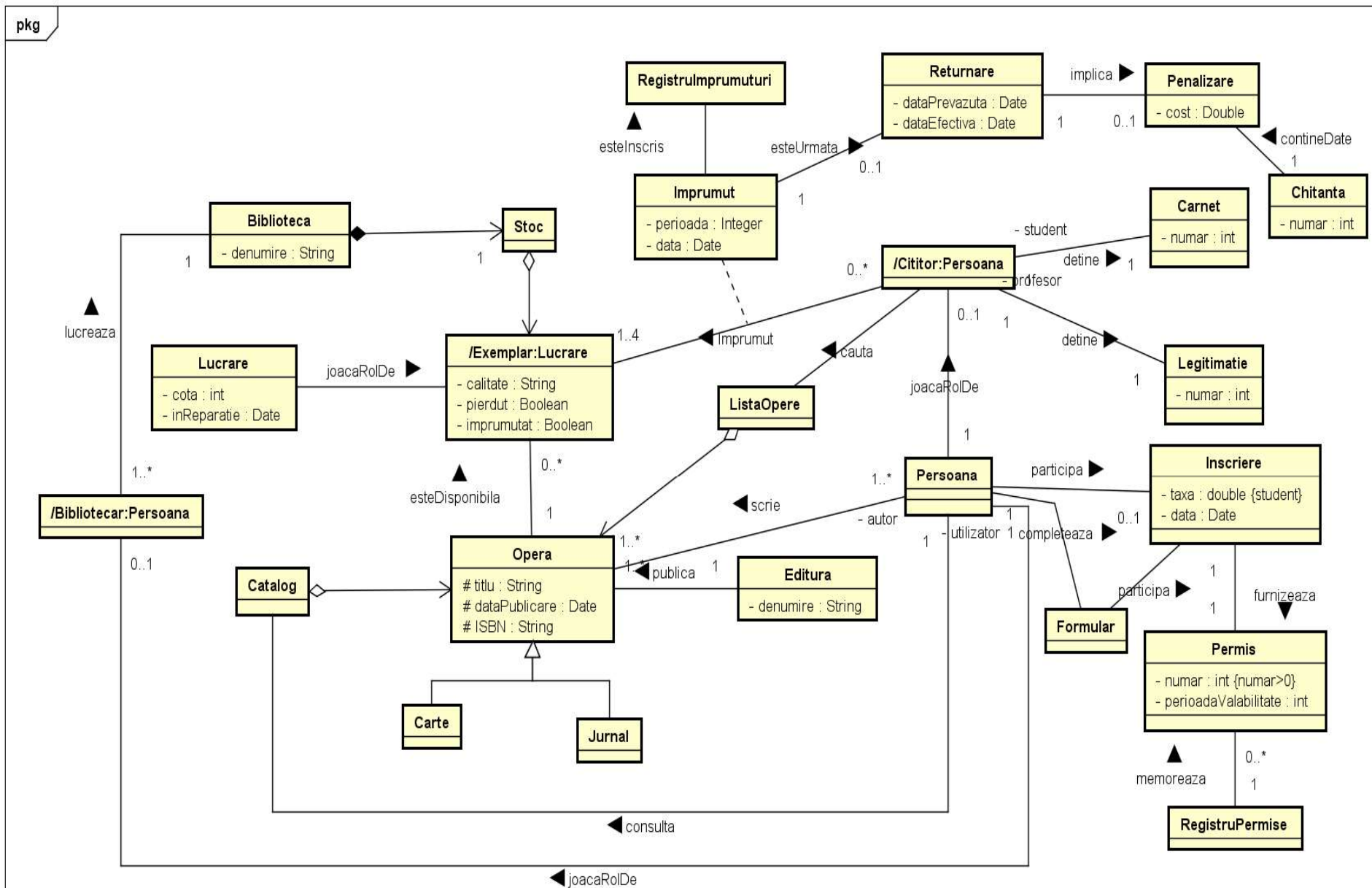
Roluri materiale => clase sau interfete UML

Roluri formale => roluri de asociere pe asocieri ale clasei Persoana cu alte clase

- Pas 5. Realizarea diagramei UML de clase a domeniului problemei
 - - clasele nu au operatii si contin numai attribute intrinseci!
 - - asocierile dintre clase nu au directie de navigare /cunoastere !



Diagrama de clase - EBiblioteca



Proiectare software



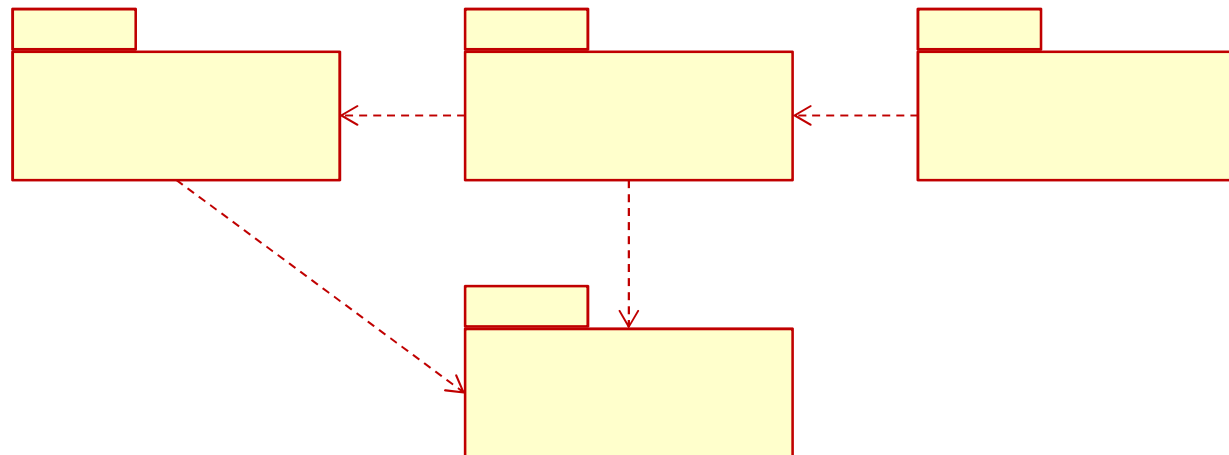
Analiza vs proiectare

- ☐ Domeniul problemei vs domeniul solutiei
- ☐ Analiza - Ce?, Proiectarea – Cum?
- ☐ In timpul proiectarii se construiesc o solutie conceptuala care sa indeplineasca cerintele functionale si nefunctionale (e.g. performanta, flexibilitate, intretinere, etc.) identificate si descrise in faza de analiza.
- ☐ Modelul domeniului problemei constituie baza proiectarii bazei de date.
- ☐ Proiectarea se imparte in doi pasi:
 - ☐ Proiectarea arhitecturala: domeniul este partitionat functional. Se stabilesc tehnologiile ce vor fi folosite pentru obtinerea unui sistem de calitate
 - ☐ Proiectarea obiectuala: sunt adaugate noi clase pentru a suporta functionalitatea asteptata de cazurile de utilizare software. Clasele sunt detaliate pentru a putea fi implementate.



Proiectarea arhitecturala

- ❑ Proiectarea arhitecturala evalueaza cerintele sistemului in raport cu tehnologiile ce ofera cele mai bune cadre pentru o solutie:
 - ❑ Diagrama UML de cazuri de utilizare software este folosita ca sursa pentru partitionarea functionala,
 - ❑ Diagrama UML de clase furnizeaza resurse pentru fiecare arie functionala,
 - ❑ Diagrame UML de interactiune furnizeaza dependentele pentru partiile functionale.
- ❑ Partitiile rezultate sunt modelate intr-o diagrama UML de pachete.



Arhitectura software

O arhitectură software = descriere a componentelor ce formează sistemul software și a conexiunilor ce coordonează acțiunilor acestor componente

Sunt utile în procesul de dezvoltare a unui sistem software:

- este deseori primul artefact din proiectare ce rezolvă decizii asupra cum sunt îndeplinite cerințele sistemului;
- este artefactul cheie în obținerea unui sistem de calitate ce poate fi creat prin reutilizarea componentelor create în procesul de dezvoltare a unor sisteme similare anterioare;
- este de obicei primul artefact citit de un programator.



Principiul de proiectare Cuplare slaba

Cuplarea este o masura a cat de strans un element este legat de, are cunostinte despre sau se bazeaza pe alte elemente. Un element cu o cuplare joasa nu depinde de prea multe elemente. Aceste elemente includ clase, subsisteme, sisteme, etc.

O clasa cu o cuplare inalta se bazeaza pe multe alte clase. In general astfel de clase sunt evitate, deoarece pot produce urmatoarele probleme:

- modificarea claselor legate determina modificari locale;
- sunt mai greu de inteles;
- sunt mai greu de reutilizat deoarece necesita prezenta claselor de care este dependenta.

In limbajele OO, formele obisnuite de cuplare a doua clase *ClasaA* si *ClasaB* sunt:

- *ClasaA* are un atribut de tip *ClasaB*.
- un obiect al clasei *ClasaA* apeleaza servicii ale unui obiect al clasei *ClasaB*
- *ClasaA* are o operatie ce refera un obiect al clasei *ClasaB* sau a clasei *ClasaB* insasi.
- *ClasaA* este o subclasa directa sau indirecta a lui *ClasaB*.
- *ClasaB* este o interfata si *ClasaA* realizeaza acea interfata.



Principiul de proiectare Coeziune inalta

In termenii proiectarii pe obiecte, **coeziunea** este o masura a cat de corelate si tind spre acelasi obiective sunt responsabilitatile unei clase.

O clasa a carei responsabilitati sunt strans legate intre ele si nu puncteaza prea multe obiective este o clasa cu coeziune inalta.

O clasa cu coeziune joasa are multe lucruri nelegate intre ele sau face prea multe.

Astfel de clase sunt evitate deoarece produc urmatoarele probleme:

- sunt greu de inteles
- sunt greu de reutilizat
- sunt greu de intretinut
- sunt obiectul modificarilor mult mai des decat celelalte clase

